



Advanced Topics

Expressions, Sorts and Declarations

In Z3, expressions, sorts and declarations are called *ASTs*. ASTs are directed acyclic graphs. Every expression has a sort (aka type). The method `sort()` retrieves the sort of an expression.

```
1  x = Int('x')
2  y = Real('y')
3  print ((x + 1).sort())
4  print ((y + 1).sort())
5  print ((x >= 2).sort())
```



The function `eq(n1, n2)` returns `True` if `n1` and `n2` are the same AST. This is a structural test.

```
1  x, y = Ints('x y')
2  print (eq(x + y, x + y))
3  print (eq(x + y, y + x))
4  n = x + y
5  print (eq(n, x + y))
6  # x2 is eq to x
7  x2 = Int('x')
8  print (eq(x, x2))
9  # the integer variable x is not equal to
10 # the real variable x
11 print (eq(Int('x'), Real('x')))
```

The method `hash()` returns a hashcode for an AST node. If `eq(n1, n2)` returns `True`, then `n1.hash()` is equal to `n2.hash()`.

```
1  x = Int('x')
2  print ((x + 1).hash())
   print ((1 + x).hash())
```

```
3     print (x.sort().hash())
4
```

Z3 expressions can be divided in three basic groups: **applications**, **quantifiers** and **bounded/free variables**. Applications are all you need if your problems do not contain universal/existential quantifiers. Although we say `Int('x')` is an integer "variable", it is technically an integer constant, and internally is represented as a function application with 0 arguments. Every application is associated with a **declaration** and contains 0 or more arguments. The method `decl()` returns the declaration associated with an application. The method `num_args()` returns the number of arguments of an application, and `arg(i)` one of the arguments. The function `is_expr(n)` returns `True` if `n` is an expression. Similarly `is_app(n)` (`is_func_decl(n)`) returns `True` if `n` is an application (declaration).

```
1     x = Int('x')
2     print ("is expression: ", is_expr(x))
3     n = x + 1
4     print ("is application:", is_app(n))
5     print ("decl:          ", n.decl())
6     print ("num args:      ", n.num_args())
7     for i in range(n.num_args()):
8         print ("arg(", i, ") ->", n.arg(i))
```

Declarations have names, they are retrieved using the method `name()`. A (function) declaration has an arity, a domain and range sorts.

```
1     x = Int('x')
2     x_d = x.decl()
3     print ("is_expr(x_d):      ", is_expr(x_d))
4     print ("is_func_decl(x_d):", is_func_decl(x_d))
5     print ("x_d.name():          ", x_d.name())
6     print ("x_d.range():          ", x_d.range())
7     print ("x_d.arity():           ", x_d.arity())
8     # x_d() creates an application with 0 arguments using x_d.
9     print ("eq(x_d(), x):      ", eq(x_d(), x))
10    print ("\n")
11    # f is a function from (Int, Real) to Bool
12    f = Function('f', IntSort(), RealSort(), BoolSort())
13    print ("f.name():              ", f.name())
14    print ("f.range():              ", f.range())
15    print ("f.arity():              ", f.arity())
16    for i in range(f.arity()):
```

```

17     print ("domain(", i, "): ", f.domain(i))
18     # f(x, x) creates an application with 2 arguments using f.
19     print (f(x, x))
20     print (eq(f(x, x).decl(), f))

```

The built-in declarations are identified using their **kind**. The kind is retrieved using the method `kind()`. The complete list of built-in declarations can be found in the file `z3consts.py (z3_api.h)` in the Z3 distribution.

```

1     x, y = Ints('x y')
2     print ((x + y).decl().kind() == Z3_OP_ADD)
3     print ((x + y).decl().kind() == Z3_OP_SUB)

```

The following example demonstrates how to substitute sub-expressions in Z3 expressions.

```

1     x, y = Ints('x y')
2     f     = Function('f', IntSort(), IntSort(), IntSort())
3     g     = Function('g', IntSort(), IntSort())
4     n     = f(f(g(x), g(g(x))), g(g(y)))
5     print (n)
6     # substitute g(g(x)) with y and g(y) with x + 1
7     print (substitute(n, (g(g(x)), y), (g(y), x + 1)))

```

The function `Const(name, sort)` declares a constant (aka variable) of the given sort. For example, the functions `Int(name)` and `Real(name)` are shorthands for `Const(name, IntSort())` and `Const(name, RealSort())`.

```

1     x = Const('x', IntSort())
2     print (eq(x, Int('x')))
3
4     a, b = Consts('a b', BoolSort())
5     print (And(a, b))

```

Arrays

As part of formulating a programme of a mathematical theory of computation McCarthy proposed a *basic* theory of arrays as characterized by the select-store axioms. The expression `Select(a, i)` returns the value stored at position `i` of the array `a`; and `Store(a, i, v)` returns a new array identical to `a`, but on position `i` it contains the value `v`. In Z3Py, we can also write `Select(a, i)` as `a[i]`.

```

1  # Use I as an alias for IntSort()
2  I = IntSort()
3  # A is an array from integer to integer
4  A = Array('A', I, I)
5  x = Int('x')
6  print (A[x])
7  print (Select(A, x))
8  print (Store(A, x, 10))
9  print (simplify(Select(Store(A, 2, x+1), 2)))

```

By default, Z3 assumes that arrays are extensional over select. In other words, Z3 also enforces that if two arrays agree on all positions, then the arrays are equal.

Z3 also contains various extensions for operations on arrays that remain decidable and amenable to efficient saturation procedures (here efficient means, with an NP-complete satisfiability complexity). We describe these extensions in the following using a collection of examples. Additional background on these extensions is available in the paper *Generalized and Efficient Array Decision Procedures*

Arrays in Z3 are used to model unbounded or very large arrays. Arrays should not be used to model small finite collections of values. It is usually much more efficient to create different variables using list comprehensions.

```

1  # We want an array with 3 elements.
2  # 1. Bad solution
3  X = Array('x', IntSort(), IntSort())
4  # Example using the array
5  print (X[0] + X[1] + X[2] >= 0)
6
7  # 2. More efficient solution
8  X = IntVector('x', 3)
9  print (X[0] + X[1] + X[2] >= 0)

```

```
9     print (Sum(X) >= 0)
10
```

Select and Store

Let us first check a basic property of arrays. Suppose A is an array of integers, then the constraints $A[x] == x$, $\text{Store}(A, x, y) == A$ are satisfiable for an array that contains an index x that maps to x , and when $x == y$. We can solve these constraints.

```
1     A = Array('A', IntSort(), IntSort())
2     x, y = Ints('x y')
3     solve(A[x] == x, Store(A, x, y) == A)
```

The interpretation/solution for array variables is very similar to the one used for functions.

The problem becomes unsatisfiable/infeasible if we add the constraint $x \neq y$.

```
1     A = Array('A', IntSort(), IntSort())
2     x, y = Ints('x y')
3     solve(A[x] == x, Store(A, x, y) == A, x != y)
```

Constant arrays

The array that maps all indices to some fixed value can be specified in Z3Py using the $K(s, v)$ construct where s is a sort/type and v is an expression. $K(s, v)$ returns a array that maps any value of s into v . The following example defines a constant array containing only ones.

```

1  AllOne = K(IntSort(), 1)
2  a, i = Ints('a i')
3  solve(a == AllOne[i])
4  # The following constraints do not have a solution
5  solve(a == AllOne[i], a != 1)

```

Datatypes

Algebraic datatypes, known from programming languages such as ML, offer a convenient way for specifying common data structures. Records and tuples are special cases of algebraic datatypes, and so are scalars (enumeration types). But algebraic datatypes are more general. They can be used to specify finite lists, trees and other recursive structures.

The following example demonstrates how to declare a List in Z3Py. It is more verbose than using the SMT 2.0 front-end, but much simpler than using the Z3 C API. It consists of two phases. First, we have to declare the new datatype, its constructors and accessors. The function `Datatype('List')` declares a "placeholder" that will contain the constructors and accessors declarations. The method `declare(cname, (aname, sort)+)` declares a constructor named `cname` with the given accessors. Each accessor has an associated `sort` or a reference to the datatypes being declared. For example, `declare('cons', ('car', IntSort()), ('cdr', List))` declares the constructor named `cons` that builds a new `List` using an integer and a `List`. It also declares the accessors `car` and `cdr`. The accessor `car` extracts the integer of a `cons` cell, and `cdr` the list of a `cons` cell. After all constructors were declared, we use the method `create()` to create the actual datatype in Z3. Z3Py makes the new Z3 declarations and constants available as slots of the new object.

```

1  # Declare a List of integers
2  List = Datatype('List')
3  # Constructor cons: (Int, List) -> List
4  List.declare('cons', ('car', IntSort()), ('cdr', List))
5  # Constructor nil: List
6  List.declare('nil')
7  # Create the datatype
8  List = List.create()
9  print(is_sort(List))
10 cons = List.cons

```

```

11  car  = List.car
12  cdr  = List.cdr
13  nil  = List.nil
14  # cons, car and cdr are function declarations, and nil a constant
15  print (is_func_decl(cons))
16  print (is_expr(nil))
17
18  l1 = cons(10, cons(20, nil))
19  print (l1)
20  print (simplify(cdr(l1)))
21  print (simplify(car(l1)))
22  print (simplify(l1 == nil))

```

The following example demonstrates how to define a Python function that given a sort creates a list of the given sort.

```

1  def DeclareList(sort):
2      List = Datatype('List_of_%s' % sort.name())
3      List.declare('cons', ('car', sort), ('cdr', List))
4      List.declare('nil')
5      return List.create()
6
7  IntList      = DeclareList(IntSort())
8  RealList     = DeclareList(RealSort())
9  IntListList  = DeclareList(IntList)
10
11  l1 = IntList.cons(10, IntList.nil)
12  print (l1)
13  print (IntListList.cons(l1, IntListList.cons(l1, IntListList.nil)))
14  print (RealList.cons("1/3", RealList.nil))
15
16  print (l1.sort())

```

The example above demonstrates that Z3 supports operator overloading. There are several functions named `cons`, but they are different since they receive and/or return values of different sorts. Note that it is not necessary to use a different sort name for each instance of the sort list. That is, the expression `'List_of_%s' % sort.name()` is not necessary, we use it just to provide more meaningful names.

As described above enumeration types are a special case of algebraic datatypes. The following example declares an enumeration type consisting of three values: `red`, `green`

and `blue`.

```

1  Color = Datatype('Color')
2  Color.declare('red')
3  Color.declare('green')
4  Color.declare('blue')
5  Color = Color.create()
6
7  print (is_expr(Color.green))
8  print (Color.green == Color.blue)
9  print (simplify(Color.green == Color.blue))
10
11  # Let c be a constant of sort Color
12  c = Const('c', Color)
13  # Then, c must be red, green or blue
14  prove(Or(c == Color.green,
15           c == Color.blue,
16           c == Color.red))

```

Z3Py also provides the following shorthand for declaring enumeration sorts.

```

1  Color, (red, green, blue) = EnumSort('Color', ('red', 'green', 'blue'))
2
3  print (green == blue)
4  print (simplify(green == blue))
5
6  c = Const('c', Color)
7  solve(c != green, c != blue)

```

Mutually recursive datatypes can also be declared. The only difference is that we use the function `CreateDatatypes` instead of the method `create()` to create the mutually recursive datatypes.

```

1  TreeList = Datatype('TreeList')
2  Tree      = Datatype('Tree')
3  Tree.declare('leaf', ('val', IntSort()))
4  Tree.declare('node', ('left', TreeList), ('right', TreeList))
5  TreeList.declare('nil')
6  TreeList.declare('cons', ('car', Tree), ('cdr', TreeList))
7

```

```

8    Tree, TreeList = CreateDatatypes(Tree, TreeList)
9
10   t1 = Tree.leaf(10)
11   t11 = TreeList.cons(t1, TreeList.nil)
12   t2 = Tree.node(t11, TreeList.nil)
13   print (t2)
14   print (simplify(Tree.val(t1)))
15
16   t1, t2, t3 = Consts('t1 t2 t3', TreeList)
17
18   solve(Distinct(t1, t2, t3))

```

The following example shows how to nest algebraic datatypes with sequences

```

1    SomeType = Datatype('SomeType')
2    SomeTypeSort = DatatypeSort('SomeType')
3    SomeType.declare('nil')
4    SomeType.declare('some', ('someof', SeqSort(SomeTypeSort), ))
5    SomeTypeSort = SomeType.create()

```

It illustrates how to access the sort `SomeTypeSort` before it is fully constructed by using the function `DatatypeSort`.

Uninterpreted Sorts

Function and constant symbols in pure first-order logic are uninterpreted or free, which means that no a priori interpretation is attached. This is in contrast to arithmetic operators such as $+$ and $-$ that have a fixed standard interpretation. Uninterpreted functions and constants are maximally flexible; they allow any interpretation that is consistent with the constraints over the function or constant.

To illustrate uninterpreted functions and constants let us introduce an (uninterpreted) sort A , and the constants x, y ranging over A . Finally let f be an uninterpreted function that takes one argument of sort A and results in a value of sort A . The example illustrates how one can force an interpretation where f applied twice to x results in x again, but f applied once to x is different from x .

```

1  A      = DeclareSort('A')
2  x, y = Consts('x y', A)
3  f      = Function('f', A, A)
4
5  s      = Solver()
6  s.add(f(f(x)) == x, f(x) == y, x != y)
7
8  print (s.check())
9  m = s.model()
10 print (m)
11 print ("interpretation assigned to A:")
12 print (m[A])

```

The resulting model introduces abstract values for the elements in A , because the sort A is uninterpreted. The interpretation for f in the model toggles between the two values for x and y , which are different. The expression $m[A]$ returns the interpretation (universe) for the uninterpreted sort A in the model m .

Quantifiers

Z3 is can solve quantifier-free problems containing arithmetic, bit-vector, Booleans, arrays, functions and datatypes. Z3 also accepts and can work with formulas that use quantifiers. It is no longer a decision procedure for such formulas in general (and for good reasons, as there can be no decision procedure for first-order logic).

```

1  f = Function('f', IntSort(), IntSort(), IntSort())
2  x, y = Ints('x y')
3  f = ForAll([x, y], f(x, y) == 0)
4  print (f.body())
5  v1 = f.body().arg(0).arg(0)
6  print (v1)
7  print (eq(v1, Var(1, IntSort())))

```

Nevertheless, Z3 is often able to handle formulas involving quantifiers. It uses several approaches to handle quantifiers. The most prolific approach is using *pattern-based* quantifier instantiation. This approach allows instantiating quantified formulas with ground terms that appear in the current search context based on *pattern annotations*

on quantifiers. Z3 also contains a model-based quantifier instantiation component that uses a model construction to find good terms to instantiate quantifiers with; and Z3 also handles many decidable fragments.

Note that in the previous example the constants `x` and `y` were used to create quantified formulas. This is a "trick" for simplifying the construction of quantified formulas in Z3Py. Internally, these constants are replaced by bounded variables. The next example demonstrates that. The method `body()` retrieves the quantified expression. In the resultant formula the bounded variables are free. The function `Var(index, sort)` creates a bounded/free variable with the given index and sort.

```

1  f = Function('f', IntSort(), IntSort(), IntSort())
2  x, y = Ints('x y')
3  f = ForAll([x, y], f(x, y) == 0)
4  print (f.body())
5  v1 = f.body().arg(0).arg(0)
6  print (v1)
7  print (eq(v1, Var(1, IntSort())))

```

Modeling with Quantifiers

Suppose we want to model an object oriented type system with single inheritance. We would need a predicate for sub-typing. Sub-typing should be a partial order, and respect single inheritance. For some built-in type constructors, such as for `array_of`, sub-typing should be monotone.

```

1  Type      = DeclareSort('Type')
2  subtype   = Function('subtype', Type, Type, BoolSort())
3  array_of  = Function('array_of', Type, Type)
4  root      = Const('root', Type)
5
6  x, y, z   = Consts('x y z', Type)
7
8  axioms = [ ForAll(x, subtype(x, x)),
9             ForAll([x, y, z], Implies(And(subtype(x, y), subtype(y, z)),
10                                     subtype(x, z))),
11            ForAll([x, y], Implies(And(subtype(x, y), subtype(y, x)),
12                                    x == y)),
13            ForAll([x, y, z], Implies(And(subtype(x, y), subtype(x, z)),
14                                        Or(subtype(y, z), subtype(z, y))))],

```

```

15         ForAll([x, y], Implies(subtype(x, y),
16                                subtype(array_of(x), array_of(y)))),
17
18         ForAll(x, subtype(root, x))
19     ]
20 s = Solver()
21 s.add(axioms)
22 print (s)
23 print (s.check())
24 print ("Interpretation for Type:")
25 print (s.model()[Type])
26 print ("Model:")
27 print (s.model())

```

Patterns

The Stanford Pascal verifier and the subsequent Simplify theorem prover pioneered the use of pattern-based quantifier instantiation. The basic idea behind pattern-based quantifier instantiation is in a sense straight-forward: Annotate a quantified formula using a pattern that contains all the bound variables. So a pattern is an expression (that does not contain binding operations, such as quantifiers) that contains variables bound by a quantifier. Then instantiate the quantifier whenever a term that matches the pattern is created during search. This is a conceptually easy starting point, but there are several subtleties that are important.

In the following example, the first two options make sure that Model-based quantifier instantiation engine is disabled. We also annotate the quantified formula with the pattern $f(g(x))$. Since there is no ground instance of this pattern, the quantifier is not instantiated, and Z3 fails to show that the formula is unsatisfiable.

```

1  f = Function('f', IntSort(), IntSort())
2  g = Function('g', IntSort(), IntSort())
3  a, b, c = Ints('a b c')
4  x = Int('x')
5
6  s = Solver()
7  s.set(auto_config=False, mbqi=False)
8
9  s.add( ForAll(x, f(g(x)) == x, patterns = [f(g(x))]),
10         g(a) == c,
11         g(b) == c,

```

```

12         a != b )
13
14     # Display solver state using internal format
15     print (s.sexpr())
16     print (s.check())

```

When the more permissive pattern $g(x)$ is used. Z3 proves the formula to be unsatisfiable. More restrictive patterns minimize the number of instantiations (and potentially improve performance), but they may also make Z3 "less complete".

```

1  f = Function('f', IntSort(), IntSort())
2  g = Function('g', IntSort(), IntSort())
3  a, b, c = Ints('a b c')
4  x = Int('x')
5
6  s = Solver()
7  s.set(auto_config=False, mbqi=False)
8
9  s.add( ForAll(x, f(g(x)) == x, patterns = [g(x)]),
10         g(a) == c,
11         g(b) == c,
12         a != b )
13
14     # Display solver state using internal format
15     print (s.sexpr())
16     print (s.check())

```

Some patterns may also create long instantiation chains. Consider the following assertion.

```

ForAll([x, y], Implies(subtype(x, y),
                        subtype(array_of(x), array_of(y))),
       patterns=[subtype(x, y)])

```

The axiom gets instantiated whenever there is some ground term of the form `subtype(s, t)`. The instantiation causes a fresh ground term `subtype(array_of(s), array_of(t))`, which enables a new instantiation. This undesirable situation is called a matching loop. Z3 uses many heuristics to break matching loops.

Before elaborating on the subtleties, we should address an important first question. What defines the terms that are created during search? In the context of most SMT solvers, and of the Simplify theorem prover, terms exist as part of the input formula, they are of course also created by instantiating quantifiers, but terms are also implicitly created when equalities are asserted. The last point means that terms are considered up to congruence and pattern matching takes place modulo ground equalities. We call the matching problem **E-matching**. For example, if we have the following equalities:

```

1  f = Function('f', IntSort(), IntSort())
2  g = Function('g', IntSort(), IntSort())
3  a, b, c = Ints('a b c')
4  x = Int('x')
5
6  s = Solver()
7  s.set(auto_config=False, mbqi=False)
8
9  s.add( ForAll(x, f(g(x)) == x, patterns = [f(g(x))]),
10         a == g(b),
11         b == c,
12         f(a) != c )
13
14  print (s.check())

```

The terms $f(a)$ and $f(g(b))$ are equal modulo the equalities. The pattern $f(g(x))$ can be matched and x bound to b and the equality $f(g(b)) == b$ is deduced.

While E-matching is an NP-complete problem, the main sources of overhead in larger verification problems comes from matching thousands of patterns in the context of an evolving set of terms and equalities. Z3 integrates an efficient E-matching engine using term indexing techniques.

Multi-patterns

In some cases, there is no pattern that contains all bound variables and does not contain interpreted symbols. In these cases, we use multi-patterns. In the following example, the quantified formula states that f is injective. This quantified formula is annotated with the multi-pattern `MultiPattern(f(x), f(y))`.

```

1  A = DeclareSort('A')
2  B = DeclareSort('B')

```

```

3  f = Function('f', A, B)
4  a1, a2 = Consts('a1 a2', A)
5  b      = Const('b', B)
6  x, y   = Consts('x y', A)
7
8  s = Solver()
9  s.add(a1 != a2,
10        f(a1) == b,
11        f(a2) == b,
12        ForAll([x, y], Implies(f(x) == f(y), x == y),
13                      patterns=[MultiPattern(f(x), f(y))])
14        )
15  print (s.check())

```

The quantified formula is instantiated for every pair of occurrences of f . A simple trick allows formulating injectivity of f in such a way that only a linear number of instantiations is required. The trick is to realize that f is injective if and only if it has a partial inverse.

```

1  A = DeclareSort('A')
2  B = DeclareSort('B')
3  f = Function('f', A, B)
4  finv = Function('finv', B, A)
5  a1, a2 = Consts('a1 a2', A)
6  b      = Const('b', B)
7  x, y   = Consts('x y', A)
8
9  s = Solver()
10 s.add(a1 != a2,
11        f(a1) == b,
12        f(a2) == b,
13        ForAll(x, finv(f(x)) == x)
14        )
15  print (s.check())

```

Other attributes

In Z3Py, the following additional attributes are supported: **qid** (quantifier identifier for debugging), **weight** (hint to the quantifier instantiation module: "more weight equals less instances"), **no_patterns** (expressions that should not be used as patterns, **skid** (identifier prefix used to create skolem constants/functions).

Multiple Solvers

In Z3Py and Z3 multiple solvers can be simultaneously used. It is also very easy to copy assertions/formulas from one solver to another.

```
1  x, y = Ints('x y')
2  s1 = Solver()
3  s1.add(x > 10, y > 10)
4  s2 = Solver()
5  # solver s2 is empty
6  print (s2)
7  # copy assertions from s1 to s2
8  s2.add(s1.assertions())
9  print (s2)
```

Unsat Cores and Soft Constraints

Z3Py also supports *unsat core extraction*. The basic idea is to use *assumptions*, that is, auxiliary propositional variables that we want to track. Assumptions are also available in the Z3 SMT 2.0 frontend, and in other Z3 front-ends. They are used to extract unsatisfiable cores. They may be also used to "retract" constraints. Note that, assumptions are not really *soft constraints*, but they can be used to implement them.

```

1  p1, p2, p3 = Bools('p1 p2 p3')
2  x, y = Ints('x y')
3  # We assert Implies(p, C) to track constraint C using p
4  s = Solver()
5  s.add(Implies(p1, x > 10),
6        Implies(p1, y > x),
7        Implies(p2, y < 5),
8        Implies(p3, y > 0))
9  print (s)
10 # Check satisfiability assuming p1, p2, p3 are true
11 print (s.check(p1, p2, p3))
12 print (s.unsat_core())
13
14 # Try again retracting p2
15 print (s.check(p1, p3))
16 print (s.model())

```

The example above also shows that a Boolean variable (`p1`) can be used to track more than one constraint. Note that Z3 does not guarantee that the unsat cores are minimal.

Formatter

Z3Py uses a formatter (aka pretty printer) for displaying formulas, expressions, solvers, and other Z3 objects. The formatter supports many configuration options. The command `set_option(html_mode=False)` makes all formulas and expressions to be displayed in Z3Py notation.

```

1  x = Int('x')
2  y = Int('y')
3  print (x**2 + y**2 >= 1)
4  set_option(html_mode=False)
5  print (x**2 + y**2 >= 1)

```

By default, Z3Py will truncate the output if the object being displayed is too big. Z3Py uses `...` to denote the output is truncated. The following configuration options can be set to control the behavior of Z3Py's formatter:

Parameter	Description
max_depth	Maximal expression depth. Deep expressions are replaced with
max_args	Maximal number of arguments to display per node.
rational_to_decimal	Display rationals as decimals if True.
precision	Maximal number of decimal places for numbers being displayed in decimal notation.
max_lines	Maximal number of lines to be displayed.
max_width	Maximal line width (this is a suggestion to Z3Py).
max_indent	Maximal indentation.

 [Edit this page](#)